

Kalman Down the Noise: Teaching Bayes What to Trust

Logan Prosser · Brian de Oliveira · Morgan Stuart
Cornell University | Reimplementation of Blundell et al. (2015)

1 Introduction

Standard neural networks learn a single point estimate for each weight, producing predictions without measure of uncertainty. **Bayes by Backprop (BBB)**, introduced by Blundell in Weight Uncertainty in Neural Networks, addresses this by treating every weight as a learned probability distribution. During each forward pass the network samples a weight configuration from those distributions, which gives us a uncertainty-aware predictions without dramatic architectural changes beyond the training objective.

The paper’s primary contributions are (1) a tractable variational objective via the negative ELBO that balances data fit against KL-divergence from a prior, (2) the reparameterization trick enabling low-variance gradient estimation through the sampling operation, and (3) thorough real tests, that BBB performs as well as or better than standard dropout on tasks like MNIST digit recognition and regression prediction problems. Our project re-implements BBB on MNIST, validates its relative performance against standard MLP and dropout baselines, and proposes a novel **gradient-trust extension** based on the **Kalman filter** of physics that damps unstable gradients before the Adam update step.

2 Chosen Result

We targeted the paper’s main MNIST classification comparison, which you can see in Figure 1 of the original work. This was the relative test-accuracy ordering of Standard MLP, Dropout MLP, and BBB with a scale-mixture prior. The result is necessary to the paper’s claim that a Bayesian treatment of weights is competitive with, or occasionally better than, dropout regularization. Getting the same ranking of results, even if the exact numbers differ, still matters because it shows the method is behaving the same way under a smaller model. In other words, BBB still seems to work for the same underlying reasons, not just because of the exact settings used in the original paper.

3 Methodology

Probabilistic weight model. Each weight stores a mean μ and a uncertainty parameter transformed by softplus ρ such that $\sigma = \ln(1 + e^\rho) > 0$. Weights are sampled via the reparameterization trick: $w = \mu + \sigma\epsilon$, where $\epsilon \sim N(0,1)$, which lets the model update μ and ρ smoothly during training instead of breaking the gradient calculation when random samples are drawn.

Loss function. The variational free-energy objective is:

$$F(D, \theta) = KL[q(w|\theta) \parallel P(w)] - \mathbb{E}_s[\ln P(D|w)]$$

The KL term penalizes distributions that change from the prior. The log-likelihood term is approximated by a single weight sample per mini-batch. We use a scale-mixture Gaussian prior which matches the original paper.

Architecture. Due to compute constraints we trained a two-hidden-layer MLP with 400 units and ReLU activations. This is smaller than the original 784 -> 1200 -> 1200 -> 10 benchmark, across four model variants: Standard MLP, Dropout MLP ($p = 0.5$), BBB, and BBB + Dropout. All models were trained on standard MNIST and evaluated on both clean MNIST and noisy FashionMNIST to verify robustness.

Kalman gradient-trust extension. We augmented the optimizer with a per-parameter Kalman inspired trust value that weights gradient updates when the running estimate based off of how uncertain gradient values are. Before each Adam step the raw gradient is multiplied by a “kizz coefficient” (trust factor) derived from a Kalman gain, which effectively dampens noisy updates without changing the base learning rate. Each of the four model variants was also trained with this Kalman modifier, which resulted in eight configurations total. Mean test accuracy and standard deviation were computed across four random seeds.

4 Results & Analysis

Reproduction fidelity. Despite using a smaller architecture, our models reproduce the paper’s relative performance ordering on clean MNIST. The Standard MLP achieved 98.12 % (paper: 98.64 %), Dropout MLP 98.10 % (paper: 98.58 %), and BBB 98.01 % (paper: 97.76 %). The gap of roughly 0.5 pp can be explained largely to reduced model capacity. The ordering is preserved and the training scenario mirror the ones that were reported, which means the implementation of the our variational objective is correct.

Kalman extension – clean MNIST. Adding the Kalman trust modifier improved the Standard MLP to $97.93\% \pm 0.10\%$, making it the strongest single model on clean MNIST. BBB variants clustered tightly between 98.02 % and 98.18 % with and without Kalman, which seems to indicate the uncertainty already captured in the weight distributions kind of overlaps with the signal the Kalman filter captures.

Kalman extension - noisy FashionMNIST. The results are more convincing when the model is tested on data that is different from what it saw during training. BBB + Dropout + Kalman achieved the highest mean accuracy ($87.52\% \pm 0.09\%$) with relevantly smaller variance than the BBB + Dropout baseline ($87.62\% \pm 0.27\%$), suggesting the Kalman dampening reduces sensitivity to noise instead of strictly improving mean accuracy.

Computational cost. A big issue with our proposed algorithm is that the Kalman modifier roughly doubles memory usage and adds approximately 35-40 % training time. In an accuracy vs cost plot, Kalman models occupy the upper-right edge, and competitive accuracy at substantially higher cost, limiting true practical applied value unless robustness is the objective.

Discrepancies. Our BBB accuracy slightly exceeds the paper’s reported value (98.01 % vs. 97.76 %). We can mostly explain this to our smaller network (fewer parameters reduce over-fitting likelihood) and to differences in mini-batch KL weighting. The paper does not clearly say how one training setting was adjusted over time. Our simpler choice may make the model a little better at getting answers right, but a little worse at knowing how confident it should be.

5 Reflections

Key lessons. An important thing we realized is that the reparameterization trick is conceptually simple but sensitive to implementation. Initializing ρ too negatively collapses all uncertainty early in training, while a too-positive initialization causes gradient variance to explode. We found that initializing $\rho \approx -3$ ($\sigma \approx 0.05$) and adding a warm-up phase for the KL weight was essential. Debugging was harder than in regular networks because the loss surface has both mean and variance parameters, so a bug in ρ shows up as slow convergence rather than null values for the gradients.

Kalman extension takeaways. The Kalman idea is most valuable in high-noise situations. On clean data, the modest accuracy gains do not justify doubled memory by any means. We would be interested in future work testing Kalman trust on harder benchmarks (CIFAR-10, corrupted datasets) where gradient noise is more severe, and could potentially try testing out layer-wise rather than per-parameter trust to reduce memory issues.

Future directions. **1**, Try it on image models: Test BBB on a LeNet-style model to see if the method still works well for more realistic image recognition. **2**, Make the confidence update smarter: Right now, the model uses a simple adjustment rule. A more advanced version could better account for how different weights move together. **3**, Let the model adapt its assumptions: Instead of using one fixed assumption about the weights, the model could learn different assumptions depending on the input (heteroscedastic architecture), which may help when test data looks different from training data. **4**, Run the bigger original model: With more computing power, we could test the full-size model from the original paper to see whether our results come from the smaller model or from our implementation choices.

References

- [1] C. Blundell, J. Cornebise, K. Kavukcuoglu, and D. Wierstra. "Weight Uncertainty in Neural Networks." Proceedings of the 32nd International Conference on Machine Learning (ICML), Lille, France, 2015, vol. 37, pp. 1613–1622.